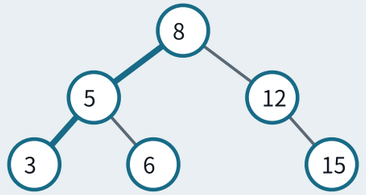
数据结构 (C++语言版) 第3版

第8章 · Advanced Search Trees

高级搜索树

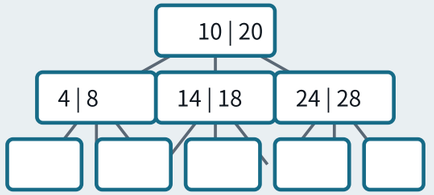
从内存到外存，从单次最优到分摊最优

探索适应不同应用场景的平衡二叉搜索树变种：**访问模式、存储介质与数据维度**决定树的结构。



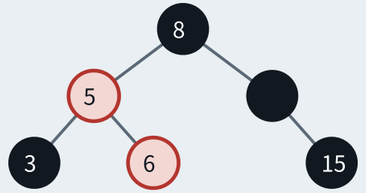
伸展树

访问路径自调整



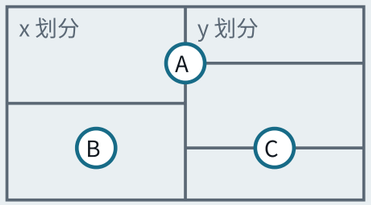
B-树

多路分支降低树高



红黑树

红黑规则适度平衡



kd-树

交替维度划分空间

来源：第8章，PDF第1页

本章议程

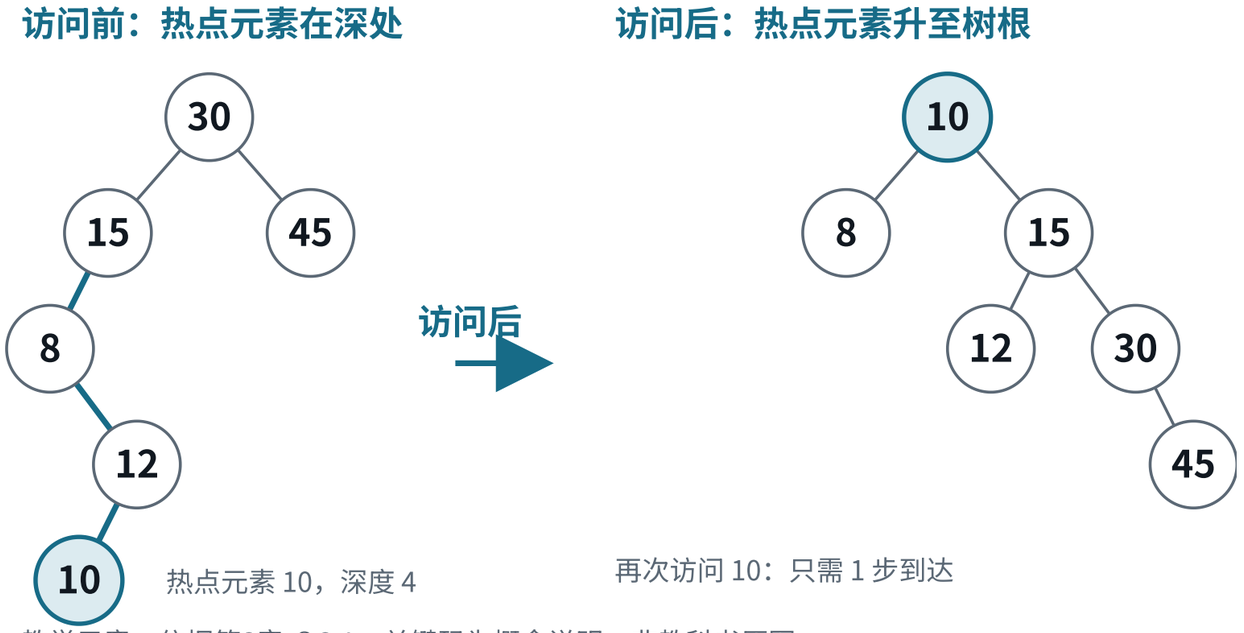
伸展树 Splay Tree	→	**数据局部性与分摊复杂度**
B-树 B-Tree	→	**分级存储与多路搜索**
红黑树 Red-Black Tree	→	**与B-树的等价性**
kd-树 kd-Tree	→	**多维范围查询**

学习目标 掌握针对特定访问模式（局部性）、存储介质（磁盘I/O）和数据维度（多维空间）的搜索树设计策略。

来源：第8章，PDF第1页

数据局部性与自调整

刚被访问的元素，值得被移到靠近树根的位置。



教学示意，依据第8章 §8.1；关键码为概念说明，非教科书原图。

数据局部性 Data Locality

① 刚被访问过的元素，极有可能在不久之后再次被访问。

② 将被访问的下一节点，极有可能处于不久之前被访问过的某个节点附近。

自调整策略：将刚被访问的节点及时“转移”至树根（附近），使热点元素更容易再次到达；转移前后搜索树必须相互等价，借助等价变换技巧实现。

两种伸展方式

- **逐层伸展**：每次只旋转提升一层，路径缩短有限。
- **双层伸展**：每次向上追溯两层并旋转，路径深度大致折半——本章重点。

来源：第8章 §8.1、§8.1.1，PDF第2页

简易伸展为何不够

单调访问序列下，每次访问的旋转次数随规模线性增长，累计得到二次方代价。

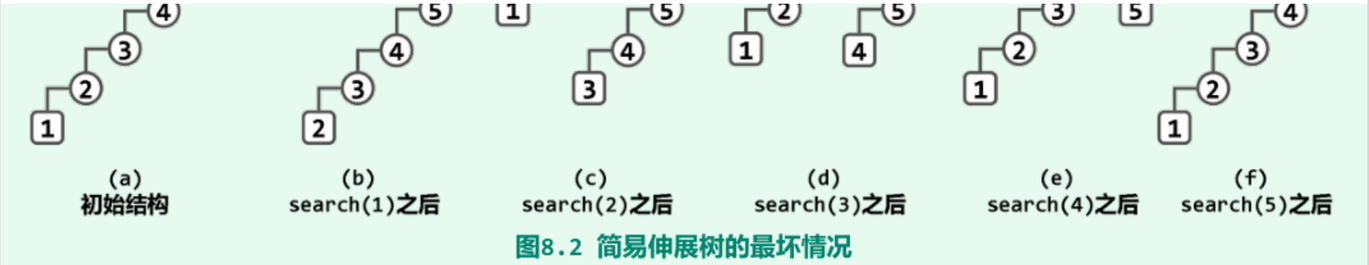


图8.2 简易伸展树的最坏情况

每次访问所需旋转次数

search(1)	4 次
search(2)	4 次
search(3)	3 次
search(4)	2 次
search(5)	1 次

一般规模 n 的累计

$$(n-1) + \{(n-1)+(n-2)+\cdots+1\}$$

$$= (n^2+n-2)/2 = \Omega(n^2)$$

分摊后每次访问平均需 Ω(n) 时间。

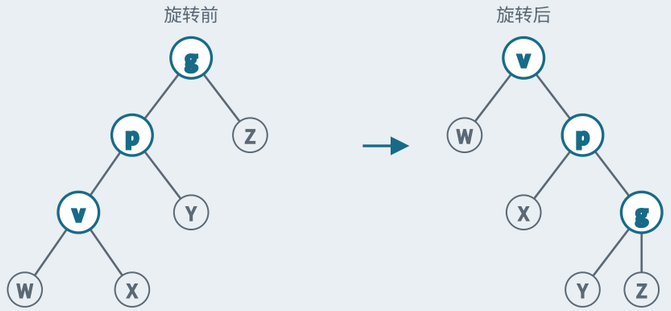
结论： 简易伸展不能保证序列意义上的对数级性能；(a) 与 (f) 结构复原，该序列可持续重复。

图8.2 简易伸展树的最坏情况（§ 8.1.2，PDF第3页）：从空树插入 {1,2,3,4,5} 得 (a)，再经 search() 由小到大依次访问，各次访问后结构为 (b)–(f)。

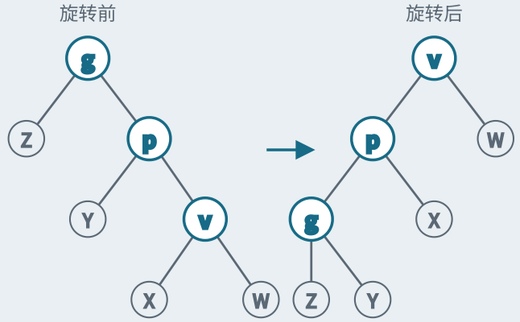
来源：第8章 § 8.1.2，PDF第3页（印刷页205），图8.2

双层伸展：把路径折半

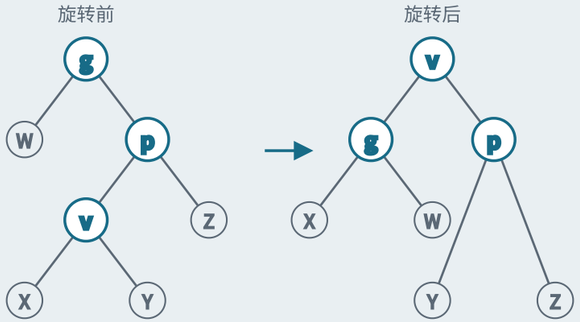
Zig-Zig 同侧双旋 · 两次顺时针



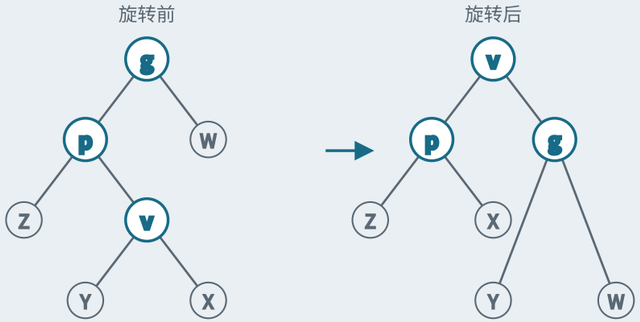
Zag-Zag 同侧双旋 · 两次逆时针



Zig-Zag 异侧双旋 · 先顺后逆



Zag-Zig 异侧双旋 · 先逆后顺



双层伸展

从目标节点 v 向上追溯两层，按父亲 p 与祖父 g 的相对位置选择旋转；每轮 v 上升两层。

同侧双旋：关键改进

zig-zig / zag-zag 两次同向旋转，v 一次上升两层，访问路径深度折半——优于逐层伸展之处。

异侧双旋

zig-zag / zag-zig 等效于 AVL 双旋，调整后的局部结构与逐层伸展一致。

单旋 zig / zag

v 深度为奇数时，最后围绕根做一次单旋；经 depth(v) 次旋转，v 总能成为树根。

效果与边界

不能杜绝最坏情况，而是控制其发生频度；单次最坏仍为 O(n)，但分摊为 O(log n)。

来源：第8章 § 8.1.3，PDF第4–5页（教学示意，依据图8.3–8.6）

双层伸展调整（一）：zig-zig 同侧双旋

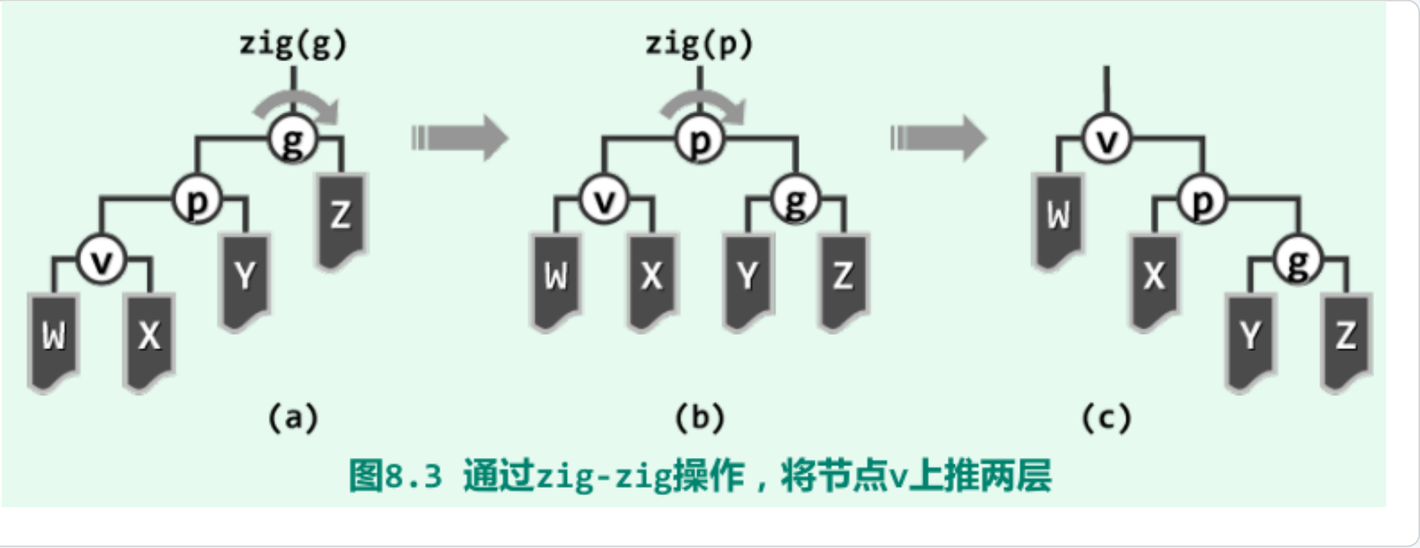


图8.3 通过 zig-zig 操作，将节点 v 上推两层（§ 8.1.3，PDF第4页）

节点关系（图 a）

v 是 p 的左孩子，p 也是 g 的左孩子——父、祖同侧。W、X 是 v 的左、右子树，Y、Z 是 p、g 的右子树。

> **第一步 zig(g)**：以 g 为轴顺时针旋转，p 取代 g 的位置（图 b）。

> **第二步 zig(p)**：再以 p 为轴顺时针旋转，v 取代 p 的位置（图 c）。

效果：v 一次上升两层，路径深度折半。

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.1.3，图8.3，PDF第4页（印刷页206）

双层伸展调整（二）：异侧双旋 zig-zag

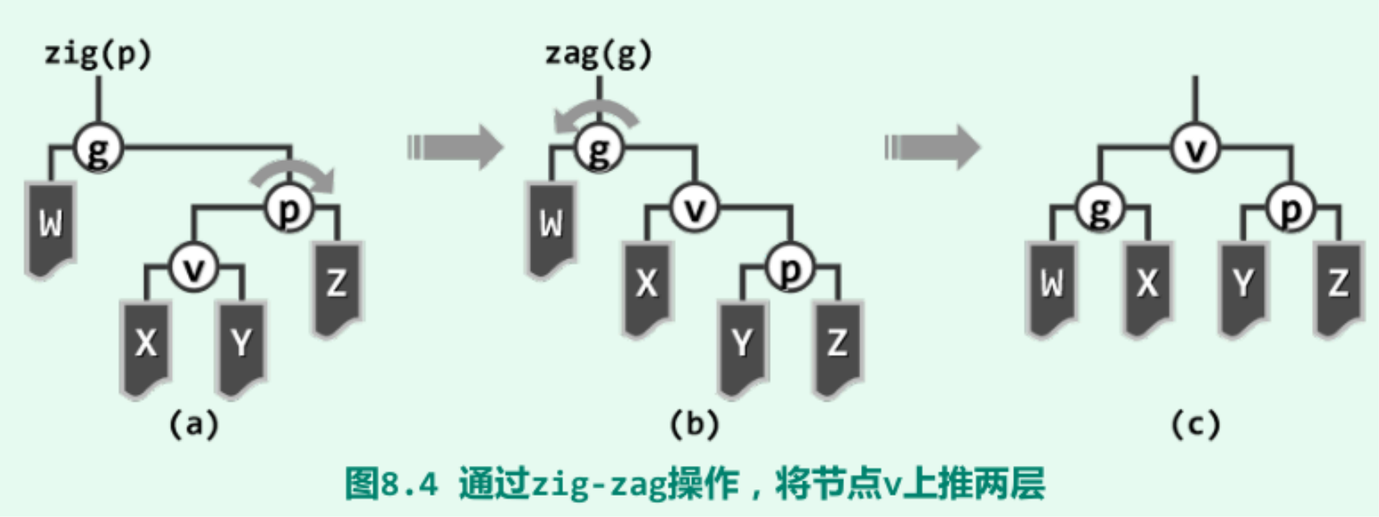


图8.4 通过zig-zag操作，将节点v上推两层

节点关系

v 是 p 的**左**孩子，p 是 g 的**右**孩子——父子位于不同侧，即异侧双旋。

调整过程

① 以 p 为轴顺时针旋转（zig(p)）
② 再以 g 为轴逆时针旋转（zag(g)）
结果：v 上升两层，成为局部子树的根。

阅读提示

对照 (a)(b)(c) 观察 v 与根的路径：双层处理使 v 一次上升两层，避免逐层伸展只缩短一步。

图8.4 通过 zig-zag 操作，将节点 v 上推两层（§ 8.1.3，PDF第4页）

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.1.3，PDF第4页，图8.4

单次最坏，序列最优

双层伸展不杜绝最坏情况，而是控制其发生频度——长路径之后的访问受益。

分摊复杂度（Amortized Complexity）

按操作序列的总代价均摊到每次操作：关注整段操作的总代价，而不是要求每次操作都短。

单次最坏情况

O(n)

一次伸展仍可能遍历整条路径，
最坏单次操作仍可能很长。
（简易伸展在单调序列下分摊即达 Ω(n)，见第4页）

双层伸展的分摊保证

O(log n)

改用双层伸展后，每次操作均可在
分摊的 O(log n) 时间内完成，
无需维护平衡因子。
（Tarjan 等人以势能分析法证明）

限定：Ω(n) 是特定可持续重复访问序列下的分摊时间，并非所有单次操作均为 Ω(n)；O(log n) 是分摊保证，不保证每次操作都短。

从内存到外存：I/O 瓶颈决定树形

内存单次访问	**纳秒级**
磁盘单次访问	**毫秒级**

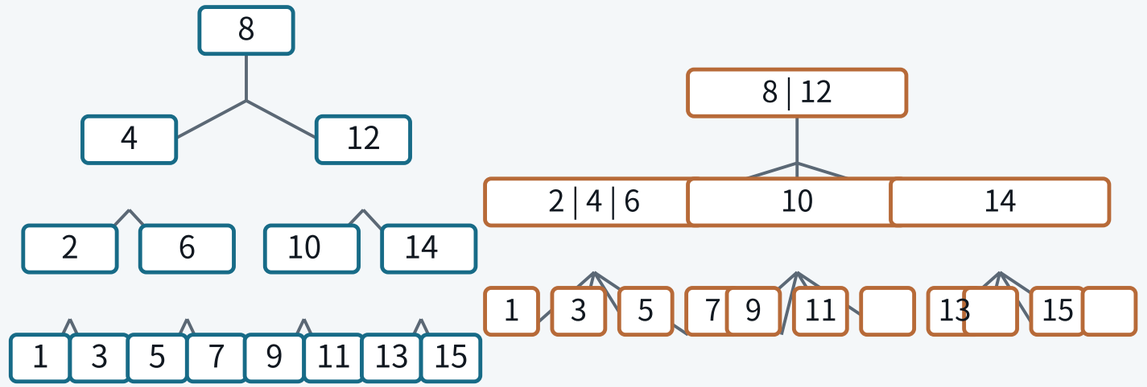
两者相差 **5 至 6 个数量级**，I/O 成为瓶颈

分级存储：数据全集存放于外存，内存作为高速缓存存放最常用数据项的复本。

查找效率主要取决于**外存访问次数**：减少页面读取次数，比减少内存中的比较次数更重要。

对策：把二叉搜索树改造为**多路搜索树**，一次 I/O 批量读出多个关键码。

教学示意（依据 §8.2.1，图8.10 的对比关系）：同样容纳 15 个关键码



二叉搜索树

每节点 1 个关键码，树高 4

四路搜索树

每节点多关键码，树高 2

宽节点、矮树 → 从根到叶的页面访问次数更少

m 阶 B-树：增加节点宽度，降低树高，减少 I/O 次数

来源：第8章 §8.2、§8.2.1，PDF第10–11页；树形对比为教学示意（对应图8.10）

B-树的宏观结构

所有外部节点深度相同、同处最底层；树高由阶数 m 与关键码数量 N 共同决定。

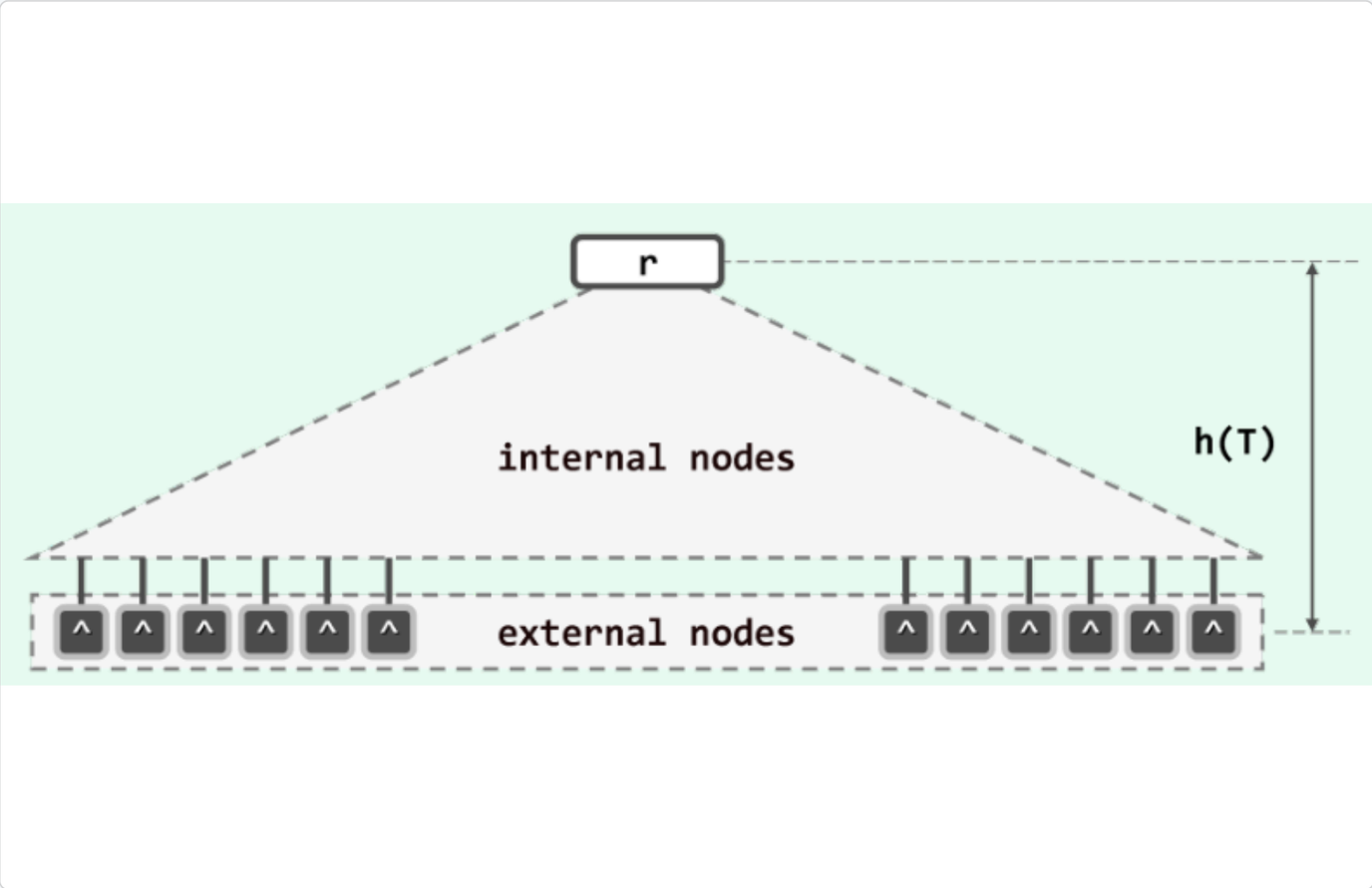


图8.11　B-树的宏观结构（附件原图）

$$h = \Theta(\log_m N)$$

树高随阶数 m 增大而下降

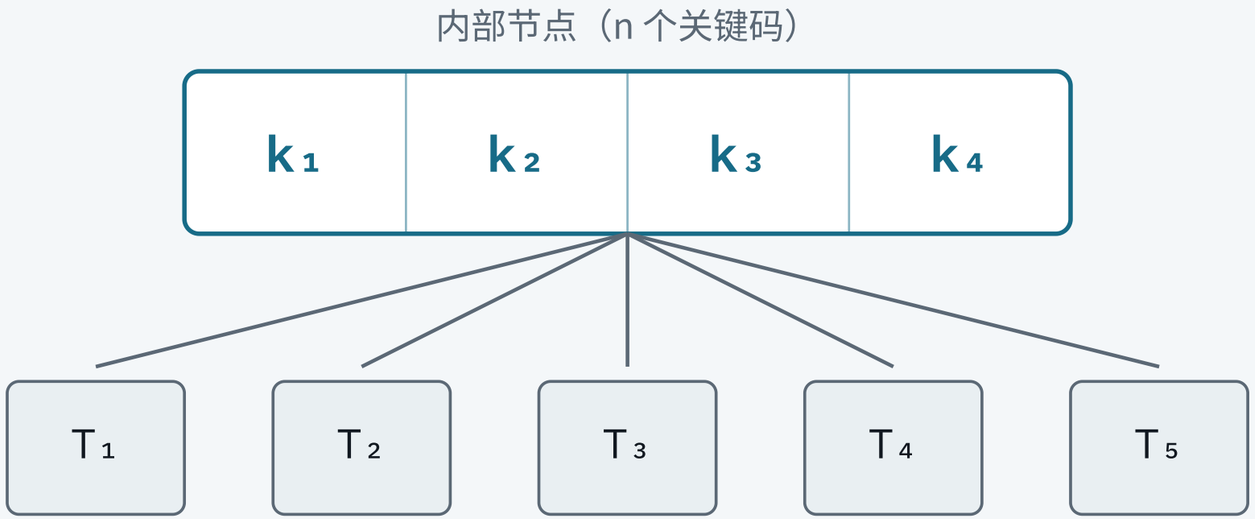
- **m**　阶数：每个节点最多 m 个分支
- **N**　关键码数量
- **h**　树高：计入最底层外部节点

m 增大 → 每节点容纳更多关键码与分支 → 树变矮。**树高决定外存页面读取次数**，降低树高即直接减少 I/O。

来源：第8章 §8.2，PDF第12页，图8.11

m阶B-树的节点约束

m阶B-树是 m 路平衡搜索树（m≥2）：节点不能任意稀疏或过满，下界保证平衡，上界决定何时分裂。



每个关键码分隔相邻两棵子树：n 个关键码 → n+1 个分支

$$\text{分支数} = n + 1 \le m$$

内部节点关键码数 n

$$\lceil m/2 \rceil - 1 \leqslant n \leqslant m-1$$

非根内部节点分支数

$$\lceil m/2 \rceil \leqslant \text{分支数} \leqslant m$$

根节点例外

非空树：分支数 ⩾ 2

根节点在建立与收缩过程中可少于一般内部节点的下界，但非空根至少有两个分支。也称 (⌈m/2⌉, m)-树，如 (2,3)-树、(3,6)-树。

教学示意：关键码与子树的一一对应（依据 §8.2）

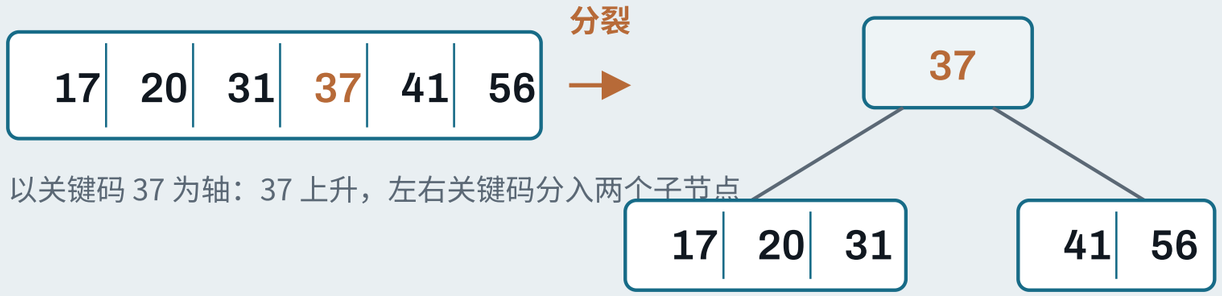
来源：《数据结构 (C++语言版) 第3版》第8章 §8.2，PDF第12页

插入上溢与节点分裂

教学示意：以图8.14的6阶B-树为例（节点 {17,20,31,37,41,56} 上溢）

分裂前
节点含 m = 6 个关键码 → 上溢

分裂后
中位数 37 上升到父节点



以关键码 37 为轴：37 上升，左右关键码分入两个子节点

左子节点 {17,20,31}，右子节点 {41,56}

分裂可向上传导：父节点饱和则继续分裂；传导至根时，提升的关键码自成新根，全树增高一层。分裂次数不超过树高，即 O(log_m N)。

查找：内存二分 + 外存读取

先在节点内二分定位关键码，再读取对应子树所在页面；效率取决于外存访问次数。

上溢（Overflow）

插入后节点关键码数**超过上限 m−1**（恰含 m 个关键码）时发生上溢。

分裂（Split）

以中位数关键码为轴：**中位数上升到父节点**，左右关键码分入两个子节点。

分裂不是删除

关键码总数不变，搜索顺序保持；若父节点也满，则继续向上分裂。

来源：《数据结构（C++语言版）第3版》第8章 §8.2，PDF第18–20页（图8.14）；教学示意按图8.14数据重绘

分裂如何修复上溢

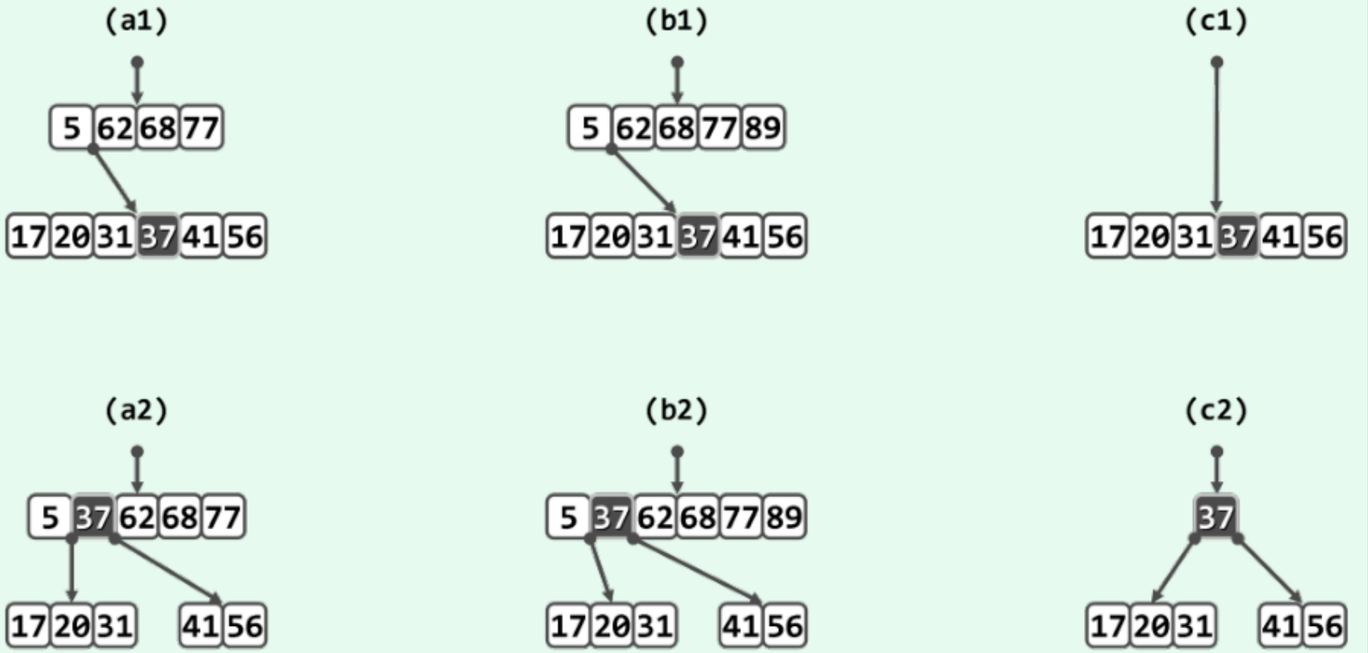


图8.14 通过分裂修复上溢节点

上溢节点

插入后关键码数超过 m−1，节点无法容纳，触发分裂。

中位数上升

中位数关键码移入父节点，作为左右子树的界。

分裂后的左右节点

原节点拆为左、右两个节点，各自重新满足容量约束。

上溢可传导

父节点因接收中位数而上溢时，分裂继续向上传递，直至根。

图8.14 通过分裂修复上溢节点（6阶B-树，节点最多5个关键码）

来源：第8章 §8.2，PDF第18页，图8.14

连续上溢的分裂过程

3阶B-树，依次插入 23、29、45：上溢如何逐级分裂、中位数如何上升

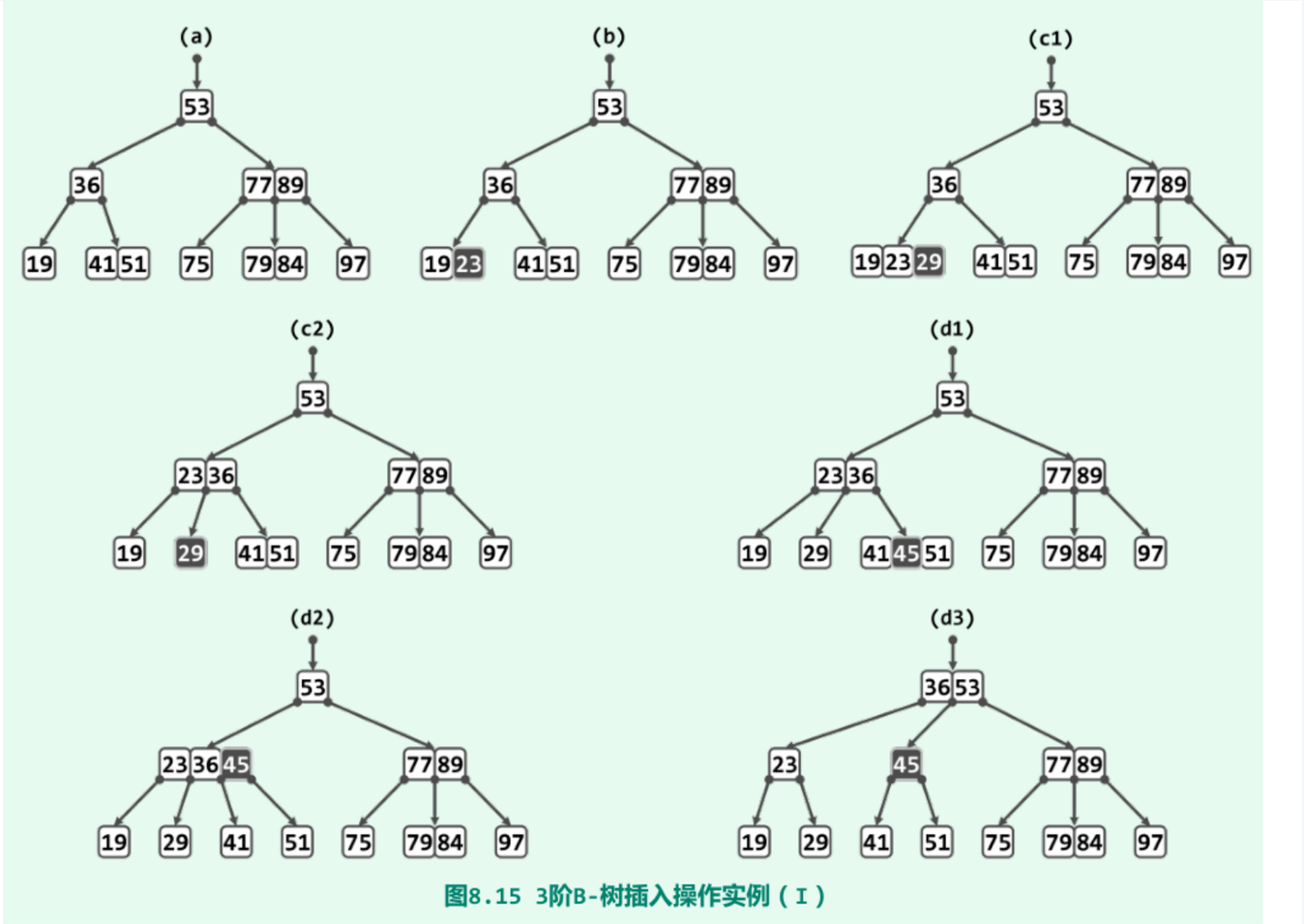


图8.15 3阶B-树插入操作实例（I）

插入 23 → 子图（b）

叶节点 {19,23,29} 未满，**未发生上溢**，直接插入。

插入 29 → 子图（c1)→(c2)

{19,23,29} 含3个关键码发生上溢，以中位数 **29** 为轴分裂，29 上升一层，**一次分裂**修复。

插入 45 → 子图（d1)→(d2)→(d3)

{41,45,51} 上溢，先以 **45** 分裂；父节点 {23,36,45} 再上溢，以 **36** 分裂，**两次分裂**修复。

最终结构 → 子图（d3）

根节点为 **{36,53}**；分裂次数不超过树高，即 O(log_m N)。

来源：第8章 §8.2，PDF第19页，图8.15

删除下溢：旋转与合并

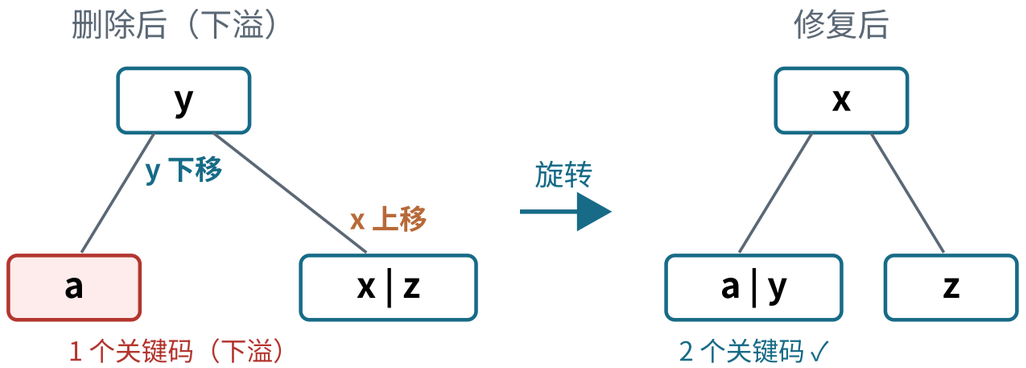
m 阶 B-树删除的两种修复方式（教学示意）

> **下溢（Underflow）**：删除后节点关键码数少于 ⌈m/2⌉−1（非根节点下界）。修复取决于兄弟节点是否富余。

旋转 Rotation

兄弟至少含 ⌈m/2⌉ 个关键码

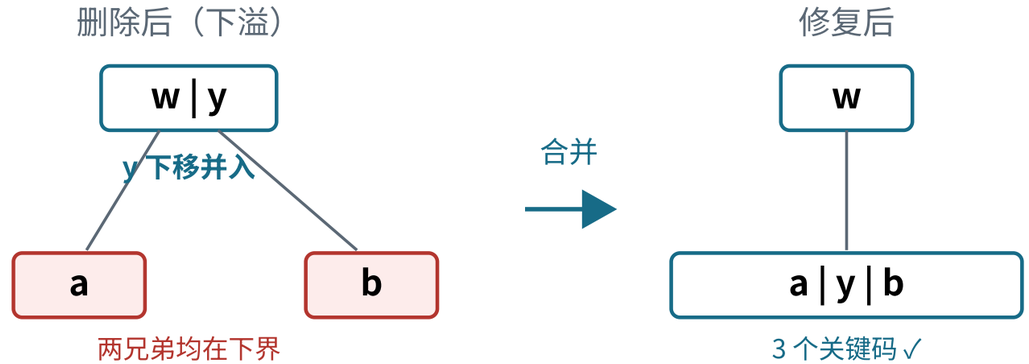
父节点关键码 y 下移，兄弟最大关键码 x 上移；树高不变。



合并 Merge

左右兄弟均无富余（各含 ⌈m/2⌉−1 个）

父节点关键码 y 下移，与兄弟合并为一个节点；节点数减少，树高可能下降。



合并后关键码数 $(\lceil m/2\rceil-1) + 1 + (\lceil m/2\rceil-2) = 2\lceil m/2\rceil-2 \le m-1$

合并后仍满足节点上界；下溢可继续向上传递，根为空时删除根，树高降一层。

来源：《数据结构 (C++语言版) 第3版》第8章 §8.2，PDF第20–22页

B-树适用场景

优化目标从内存比较次数，转向外存页面读取次数。

宽节点 · 多路分支

m 阶 B-树节点含多个关键码，一次读取即获得多路分支。

→

树高降低

高度 h 满足 $\log_m(N+1) \leqslant h \leqslant \log_{\lceil m/2 \rceil}\lfloor (N+1)/2 \rfloor + 1$，随阶数 m 增大而下降。

→

页面读取次数减少

查找每高度至多访问一个节点，外存访问不超过 O(h−1) 次；效率主要取决于外存访问次数。

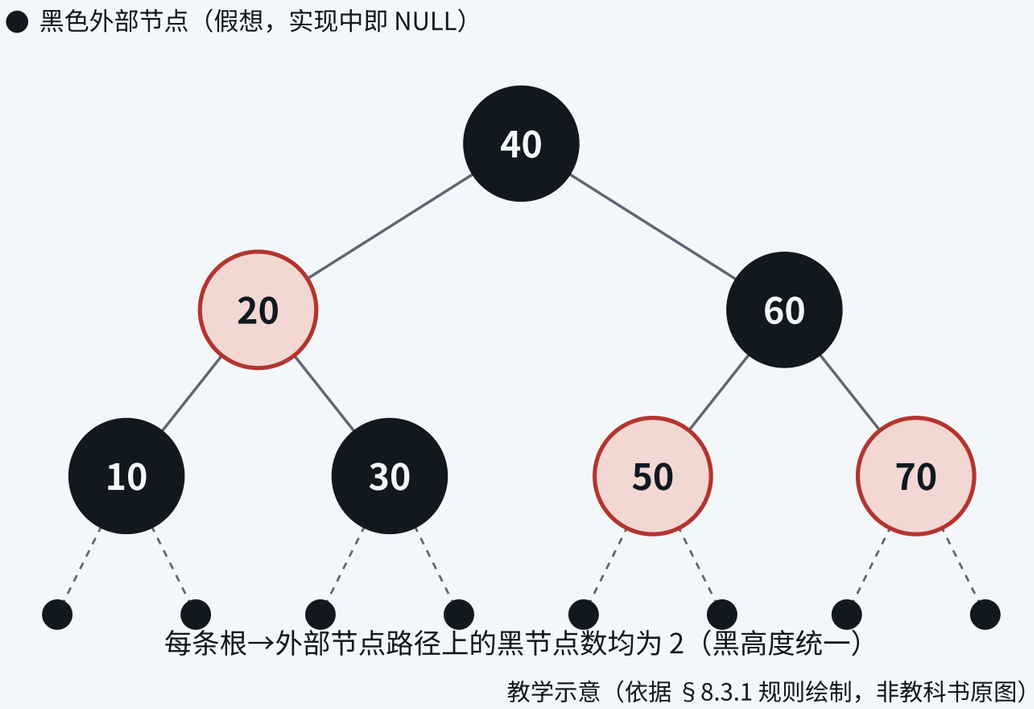
适用 数据量大、全集存放于**外存（磁盘）**、以页面为单位批量读取的分级存储场景；内存仅作高速缓存。

代价 节点内查找与分裂／合并的维护逻辑，比二叉树更复杂。

来源：第8章 §8.2，PDF第10–11、13–14页

红黑树的四条规则

动机：AVL 树删除后可能需要多达 Ω(log n) 次旋转，全树拓扑大幅变化；红黑树以红黑分布约束换取**每次插入/删除仅常数次拓扑调整**的持久性结构。



教学示意（依据 § 8.3.1 规则绘制，非教科书原图）

1 根为黑色

树根始终为黑节点。

2 外部节点为黑色

仿照 B-树假想引入黑色外部节点，使每个内部节点的左、右孩子均非空；实现中可统一将 NULL 视作黑色外部节点。

3 红节点的孩子必为黑色

不允许红—红相邻。

4 黑高度统一

从任一外部节点到根节点的沿途，黑节点数目相等。

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.3–8.3.1，PDF第26页

红黑树为何适度平衡

不是严格平衡，但树高仍被限制在对数范围内。

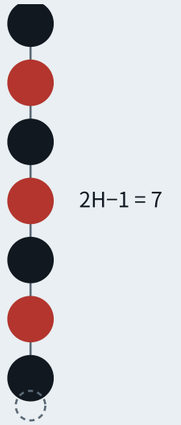
最短路径

全黑，长度 = H



最长路径

红黑交替，至多 2H−1



教学示意（依据 § 8.3.1）：规则 3 禁止相邻红节点，故最长路径在每个黑节点之间至多插入一个红节点。

1. 设全树黑高度为 **H**：根到任一外部节点路径上的黑节点数（规则 4 保证唯一）。
2. 最短路径全黑，长度恰为 **H**；最长路径红黑交替，**$h \le 2H$**。
3. 最短全黑路径上每个黑节点都带两棵子树，故节点数 **$n \ge 2^H - 1$**，即 **$H \le \log_2(n+1)$**。
4. 代入第 2 步：**$h \le 2H \le 2\log_2(n+1)$**。

$$\log_2(n+1) \le h \le 2\log_2(n+1)$$

下界对任意二叉树成立（$n \leqslant 2^h - 1$）；上界来自红黑性质。

适度平衡 允许路径长度差异，但差异被限制在对数范围内——树高始终为 O(log n)。

来源：第8章 § 8.3.1，PDF第26–27页

红黑树与4阶B-树的等价转换

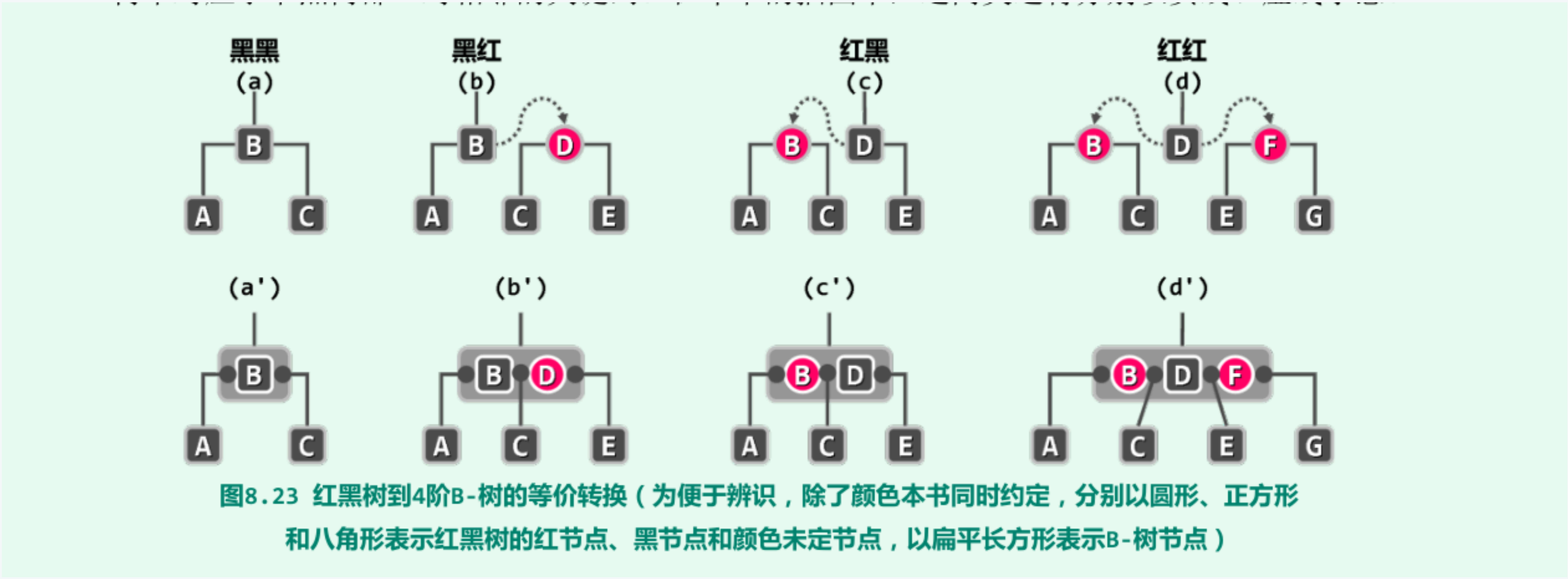


图8.23　红黑树到4阶B-树的等价转换（§8.3，PDF第27页）

提升变换

遇到**红节点**时，将其子树提升一层，与黑色父节点水平对齐，父子联边改为横向。

核心映射

红节点对应**B-树节点内部的水平连接**，不是独立B-树节点；每组至多一个黑节点加两个红节点，构成含1–3个关键码的4阶B-树节点。

图例约定

● 红节点（圆形）　● 黑节点（正方形）

实线为树边；虚线弧表示红节点的提升变换。

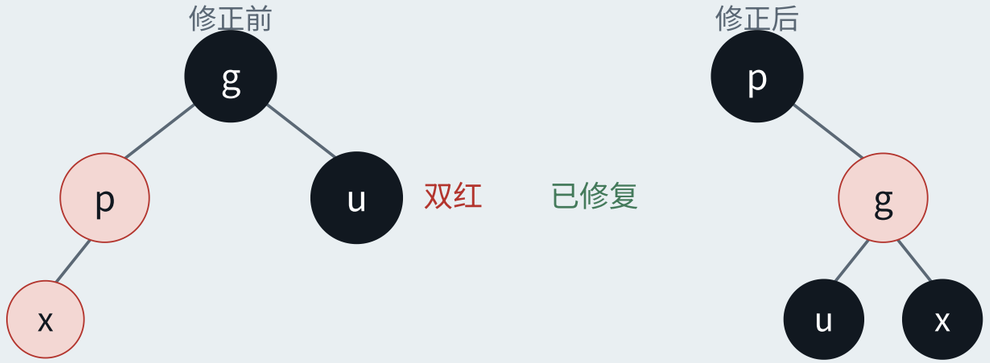
来源：《数据结构 (C++语言版) 第3版》第8章 §8.3.1，PDF第27页（印刷页229），图8.23

插入修正：处理双红

新节点先染红，插入后仅条件(3)可能违反——**父节点与子节点同为红**，称为**双红（Double Red）**。记 x 为红节点，p 为父，g 为祖父，u 为叔父；p 红时 g 必存在且为黑。

RR-1　叔父为黑：旋转 + 染色

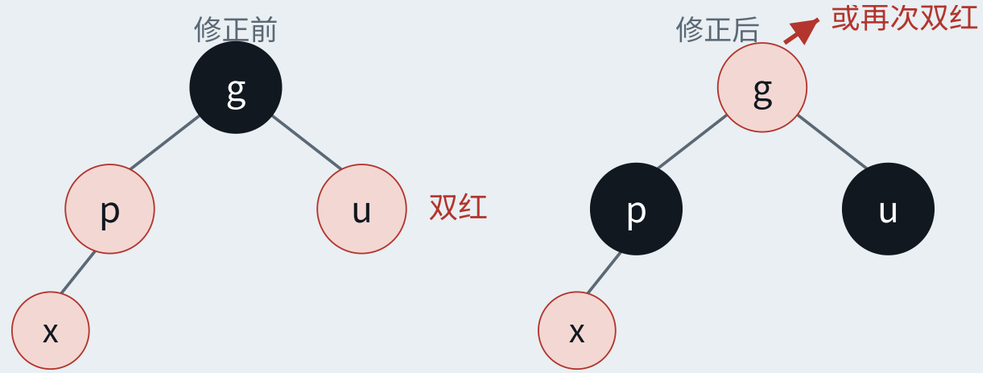
u 为黑 → 局部“3+4”重构，调整随即完成



按中序次序对 x、p、g 及四棵子树做局部重构；B-树视角即**黑关键码与相邻红关键码互换颜色**。旋转 1～2 次、染色 2 次。

RR-2　叔父为红：染色 + 上溢传导

u 为红 → 只染色，冲突向上传递



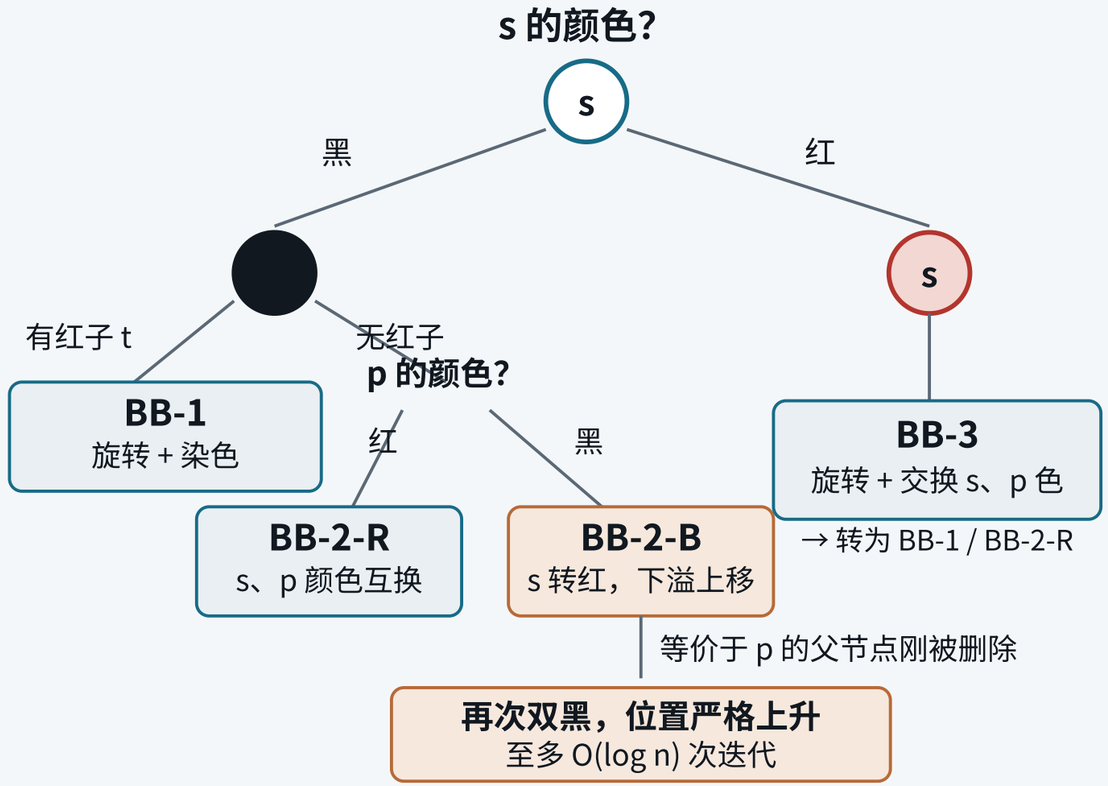
p、u 转黑，g 转红，x 保持红；局部黑高度复原，但 g 红可能在上层再次双红。每次迭代双红位置**上升两层**，至多 O(log n) 轮；若最终使原根分裂，新根 g 须转黑。

对应 B-树　RR-2 相当于 **B-树节点上溢分裂向上传导**；RR-1 相当于节点内关键码颜色互换。只有 RR-1 需要旋转，故每次插入的拓扑调整仅常数次，插入总耗时 O(log n)。

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.3.3，PDF第29–30页（图8.25–8.27，表8.1）；树形为教学示意

删除修正：处理双黑

双黑（Double Black）：删除后接替者 r 与被删节点 x 均为黑，r 一侧的黑高度比兄弟一侧少 1，需补偿。记 x 的父亲为 p、兄弟为 s，按 s 与 p 的颜色及 s 是否有红孩子分四种情形。



教学示意 · 依据 § 8.3.4（图8.28–8.33）

BB-1　黑 s 有红孩子 t

对 t、s、p 做“3+4”重构：两端染黑，中间继承 p 原色，r 保持黑。B-树：从父节点借关键码。

BB-2-R　黑 s 无红孩子，p 红

s 与 p 颜色互换，调整随即完成，不引发新下溢。B-树：父关键码下降并合并左右孩子。

BB-3　s 红（p 必黑）

以 p 为轴旋转并交换 s、p 颜色；r 获得新的黑兄弟，后续只进 BB-1 或 BB-2-R，至多再迭代一步。

BB-2-B　黑 s 无红孩子，p 黑

s 由黑转红，局部恢复，但 p 继续下溢——唯一可能再次迭代的情形。B-树：下溢节点与兄弟合并。

结论：除 BB-2-B 外均一次完成；BB-2-B 下溢位置严格上升，删除总耗时 O(log n)。

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.3.4，PDF第32–35页（图8.28–8.33、表8.2）

拓扑调整不超过常数次

旋转改变树的拓扑结构；染色只改颜色、不改变结构。下表分别统计插入与删除各情形的旋转与染色次数。

表8.1　插入修正（双红）

Table 8.1 · PDF第30页

情况	条件	旋转次数	染色次数	结果
RR-1	叔父 u 为黑	**1～2次**	**2次**	调整随即完成
RR-2	叔父 u 为红	**0次**	**3次**	或再次双红，但必上升两层

RR-2 不旋转，仅染色；双红位置每次上升两层，至多 O(log n) 轮。

表8.2　删除修正（双黑）

Table 8.2 · PDF第35页

情况	条件	旋转次数	染色次数	结果
BB-1	黑 s 有红子 t	**1～2次**	**3次**	调整随即完成
BB-2-R	黑 s 无红子、p 红	**0次**	**2次**	调整随即完成
BB-2-B	黑 s 无红子、p 黑	**0次**	**1次**	必然再次双黑，但上升一层
BB-3	s 为红	**1次**	**2次**	转为 BB-1 或 BB-2-R

BB-2-B 是唯一可能再次迭代的情形；下溢位置严格上升，至多 O(log n) 次。

结论：无论插入还是删除，拓扑结构调整（旋转）均不超过常数次。

染色可能向上传导，但旋转次数受常数限制——这正是红黑树适合持久结构的定量依据。

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.3.3–8.3.4，表8.1（PDF第30页）、表8.2（PDF第35页）

红黑树适用场景

持久结构中的常数次调整：平衡性与修改代价的折中。

插入与删除的拓扑调整（旋转）不超过常数次，结构长期保持适度平衡。

不追求每次操作的最短路径，而是保证结构长期可用、修改代价可控；为持久性结构（persistent structure）的实现提供直接而有效的方法。

适度平衡

树高受黑高度约束：$h \le 2 \cdot \log_2(n+1)$，即 O(log n)。

与 4 阶 B-树等价

插入修正对应节点分裂，删除修正对应旋转与合并；修正机制可统一理解为 B-树的上溢与下溢处理。

适用场景

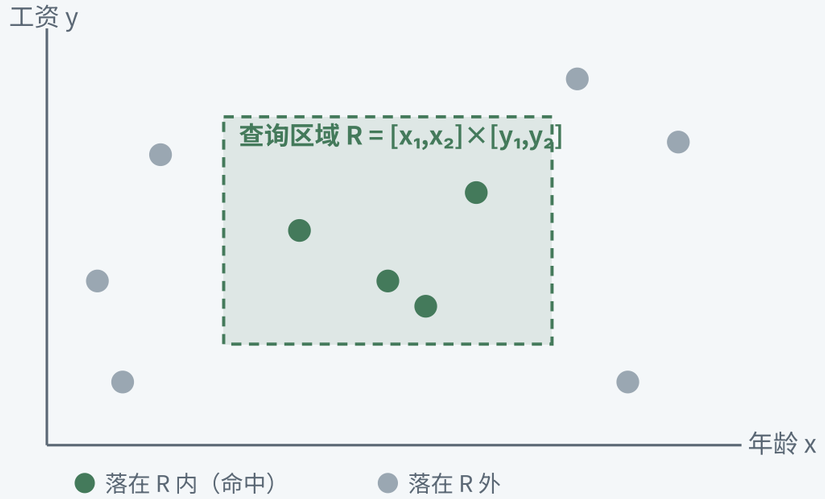
需要频繁插入/删除、且希望拓扑调整次数有常数保证的结构；单次操作效率稳定。

来源：第8章 §8.3.1–8.3.4，PDF第26–35页

多维范围查询与kd-树

一维搜索树无法同时表达两个维度的约束；kd-树轮流按不同维度切分空间，使每个子树对应一个矩形区域。

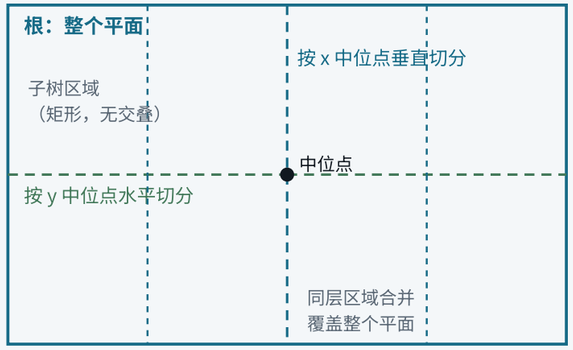
问题：平面范围查询（Planar Range Query）



教学示意，依据 § 8.4.1（PDF第37页，图8.36）：将年龄、工资表示为两个正交维度，查询区域为矩形。

> **实例**：查找年龄 20–30 且工资 10k–20k 的员工——两个条件必须**同时**满足。

结构：交替维度划分



教学示意，依据 § 8.4.2（PDF第40页，图8.39、算法8.1）：每层交替使用 x、y 维度切分。

- 每个节点对应一个与坐标轴平行的矩形区域；同层区域合并覆盖全平面且无交叠。
- 每次按对应坐标的**中位点**切分，两侧各含约一半输入点。
- 中位点切分保证树高不超过 O(log n)，树保持平衡。

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.4.1–8.4.2，PDF第37、40页

2d-树的构造与平面划分

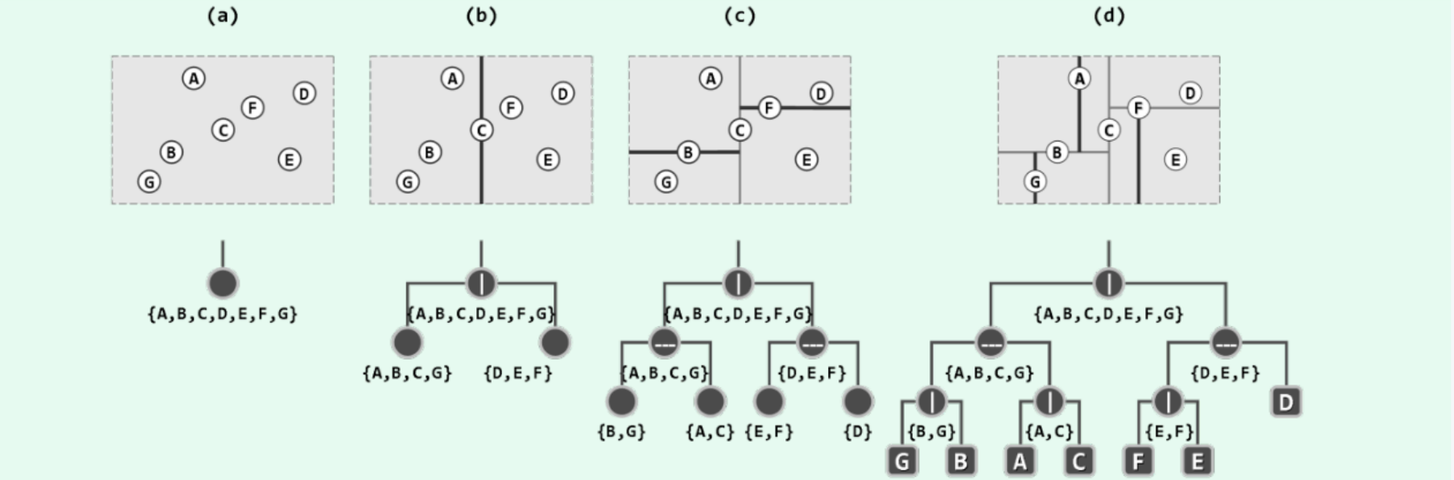


图8.40 2d-树的构造过程，就是对平面递归划分的过程

第1轮 · 水平中位点 C

根含全部7点；按水平中位点 C 切分：

左 {A,B,C,G} 右 {D,E,F}

第2轮 · 垂直中位点

左半按垂直中位点 B 切为 {B,G}、{A,C}；

右半按垂直中位点 F 切为 {E,F}、{D}。

第3轮 · 水平中位点

对仍含两点的三个节点沿水平中位点切分，

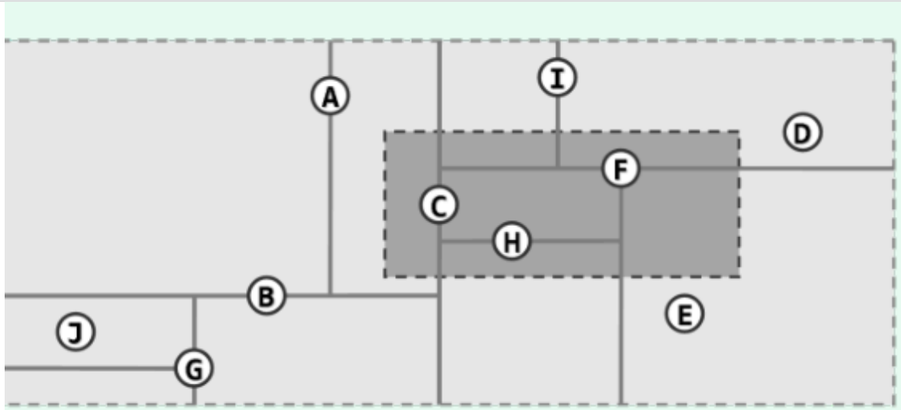
所有叶节点只含单点。

划分维度逐层交替（水平→垂直→水平）；中位点切分使左右子树规模保持平衡，每个节点对应一块矩形区域。

图8.40　2d-树的构造过程，就是对平面递归划分的过程（a→d 逐轮划分）

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.4.2，PDF第41页（印刷页243），图8.40

基于2d-树的平面范围查询



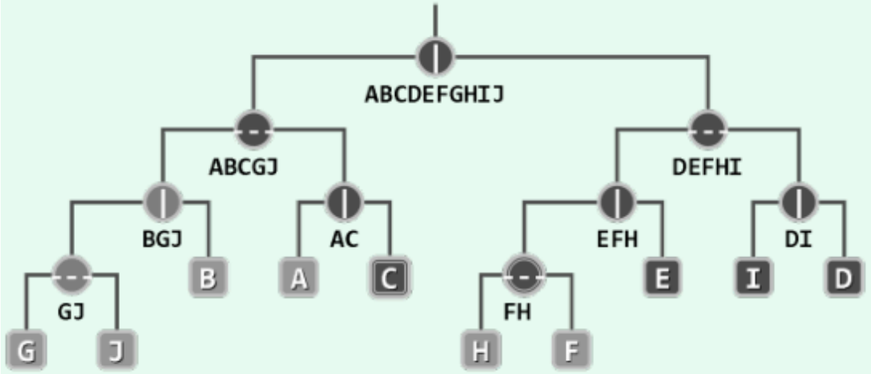


图8.41 基于2d-树的平面范围查询

A～J共计10个输入点；命中子树的根节点，以双线圆圈示意）

查询结果（落在查询矩形内的点）

{ F, H, C }

- **黑色节点 → 递归**
  子树区域与查询区域相交，继续向下查询。
- **灰色节点 → 剪枝**
  与查询区域无交集，或父节点区域已被全包含，不再递归。

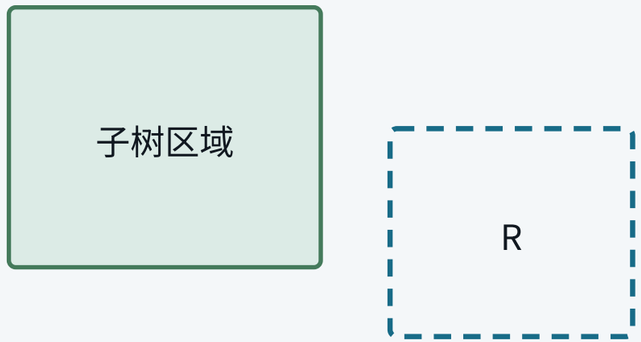
C 作为叶节点直接判断是否落在 R 内；F、H 对应区域完全包含于查询区域，经遍历直接报告。递归只发生在相交情形。

来源：第8章 §8.4.3，PDF第42页（印刷页244），图8.41

查询算法与输出敏感性

非叶节点按子树区域与查询区域 R 的关系决定动作；递归只发生在相交情形。

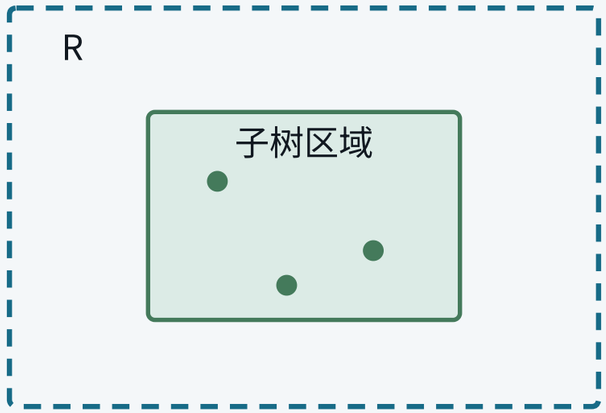
无交集



剪枝：直接返回

区域内不可能有答案，整棵子树跳过

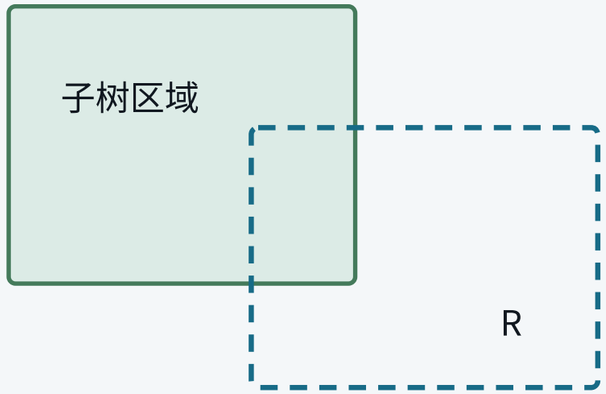
完全包含



报告：遍历该子树

区域内所有点必在 R 内，直接输出

相交



递归：检查子树

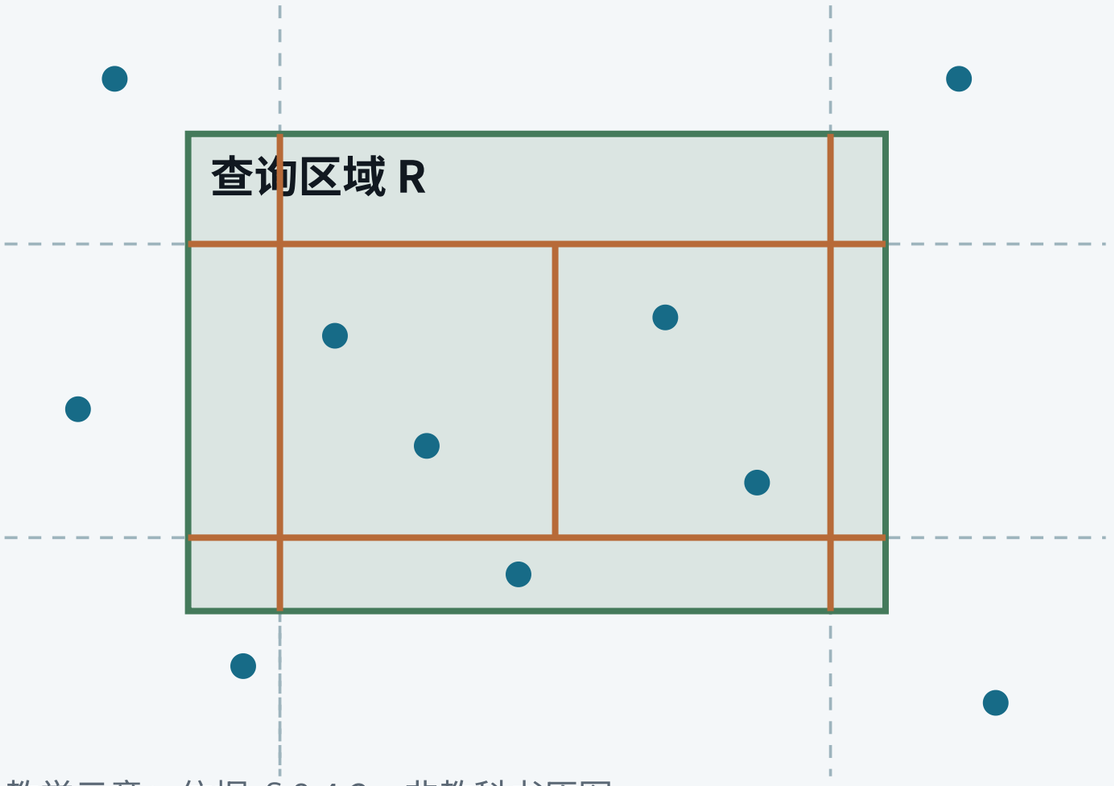
部分点可能命中，需继续向下查询

输出敏感（Output Sensitive）

查询时间**取决于输出点的数量**，而非仅由输入规模决定——结果越多，报告成本越高。

来源：第8章 §8.4.3（算法8.2），PDF第42页；区域关系为教学示意

kd-树查询复杂度



教学示意：依据 § 8.4.3，非教科书原图

查询矩形边界与划分线相交（赭色）：只有这些边界处的子树需要递归访问。

O(√n)

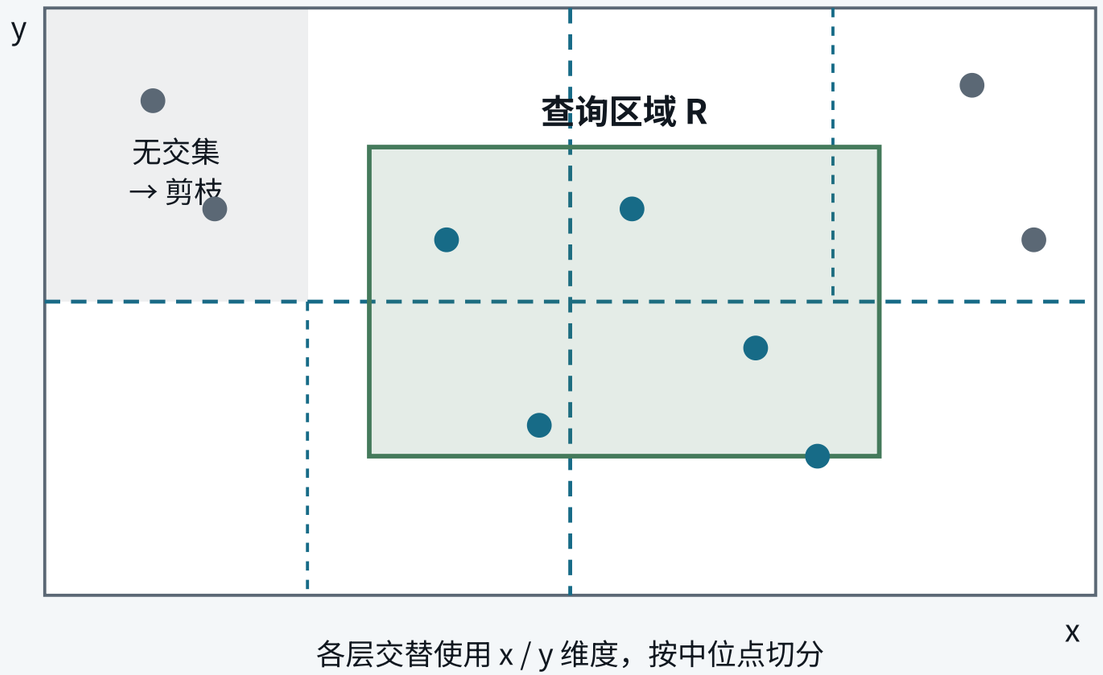
仅适用于二维（2D），不得推广到任意维度

教材结论：被报告子树总数不超过 O(√n) 棵，累计耗时 O(√n)。

- **√n 的直觉**：查询矩形边界最多与 O(√n) 条划分线相交，边界子树的访问规模受此限制。
- **输出敏感**：查询时间还取决于输出点数量 k——报告每个命中点的成本随 k 增长。
- 同一深度可能递归两次以上、报告多于两棵子树，因此不能简单套用 O(log n)。

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.4.3，PDF第42页

kd-树适用场景



教学示意：交替维度划分与区域剪枝（依据 § 8.4，非教材原图）

适用

固定点集上的多维范围查询与空间索引

是四叉树、八叉树等结构的一般性推广，为计算几何类应用提供基本模式

优势

交替维度划分，中位点切分保证树高 O(log n)

查询按区域关系剪枝：无交集直接返回，全包含直接报告

限制

查询代价与输出规模相关（输出敏感）

O(√n) 仅适用于二维；高维场景需结合具体应用评估

多维约束转化为空间区域判断：能利用区域剪枝的查询适合 kd-树，但结果规模会影响总代价。

来源：《数据结构 (C++语言版) 第3版》第8章 § 8.4（适用场景小结）

按场景选择搜索树

先问三个问题：数据怎样访问？存放在哪里？有多少维度？

结构	适用场景	关键机制与收益
伸展树 Splay Tree	**局部性强的访问**	刚访问的元素靠近根，双层伸展使序列操作分摊 O(log n)
B-树 B-Tree	**外存数据库**	多路分支降低树高，批量读取减少外存页面访问次数
红黑树 Red-Black Tree	**持久结构**	插入/删除的拓扑调整不超过常数次，单次效率稳定
kd-树 kd-Tree	**多维范围查询**	交替维度划分空间，区域无交集即剪枝，时间随输出规模变化

> **结构的目标不是单一指标最优，而是匹配实际应用场景。**

来源：第8章 §8.1–8.4，PDF第23页（§8.3开头比较性总结）、10–14页、26–35页、36–42页